

SIMULATION RESULTS

By
ShockOps
Team 1

1. Sample codes for level one (ADDI ADD SUB AND OR XOR SLL SRL SLT)

```
ADDI x2, x0, 10    # x2 = 10
ADDI x3, x0, 4     # x3 = 4
```

```
ADD  x4, x2, x3    # x4 = 14
SUB   x5, x2, x3    # x5 = 6
AND   x6, x2, x3    # x6 = 0
OR    x7, x2, x3    # x7 = 14
XOR   x1, x2, x3    # x1 = 14
```

```
SLL  x8, x2, x3     # x8 = 10 << 4 = 160
SRL  x9, x2, x3     # x9 = 10 >> 4 = 0
SLT  x10, x3, x2     # x10 = 1
```

```
00A00113
00400193
00310233
403102B3
00317333
003163B3
003140B3
00311433
003154B3
0021A533
```

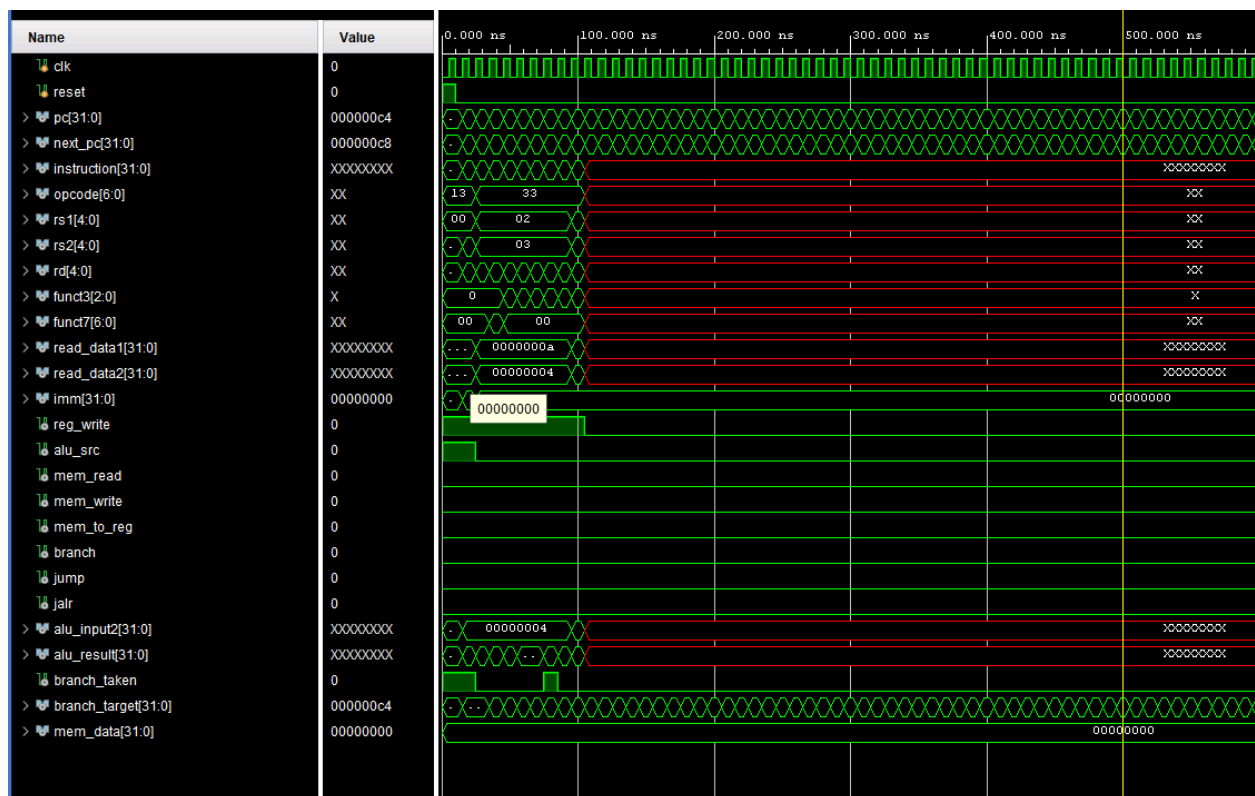
Expected Outputs:

```
x1=14
x2=10
x3=4
x4=14
x5=6
x6=0
x7=14
x8=160
x9=0
x10=1
```

Received Outputs:

FINAL REGISTER VALUES

```
=====
x1 =      14
x2 =      10
x3 =       4
x4 =      14
x5 =       6
x6 =       0
x7 =      14
x8 =     160
=====
```



2. Sample code to test **LW** and **SW** and Results.

Hex Code

```
00A00093
00000113
00112023
00012183
00118233
```

Outputs

REGISTER VALUES

```
x1 = 10
x2 = 0
x3 = 10
x4 = 20
x5 = 0
x6 = 0
```

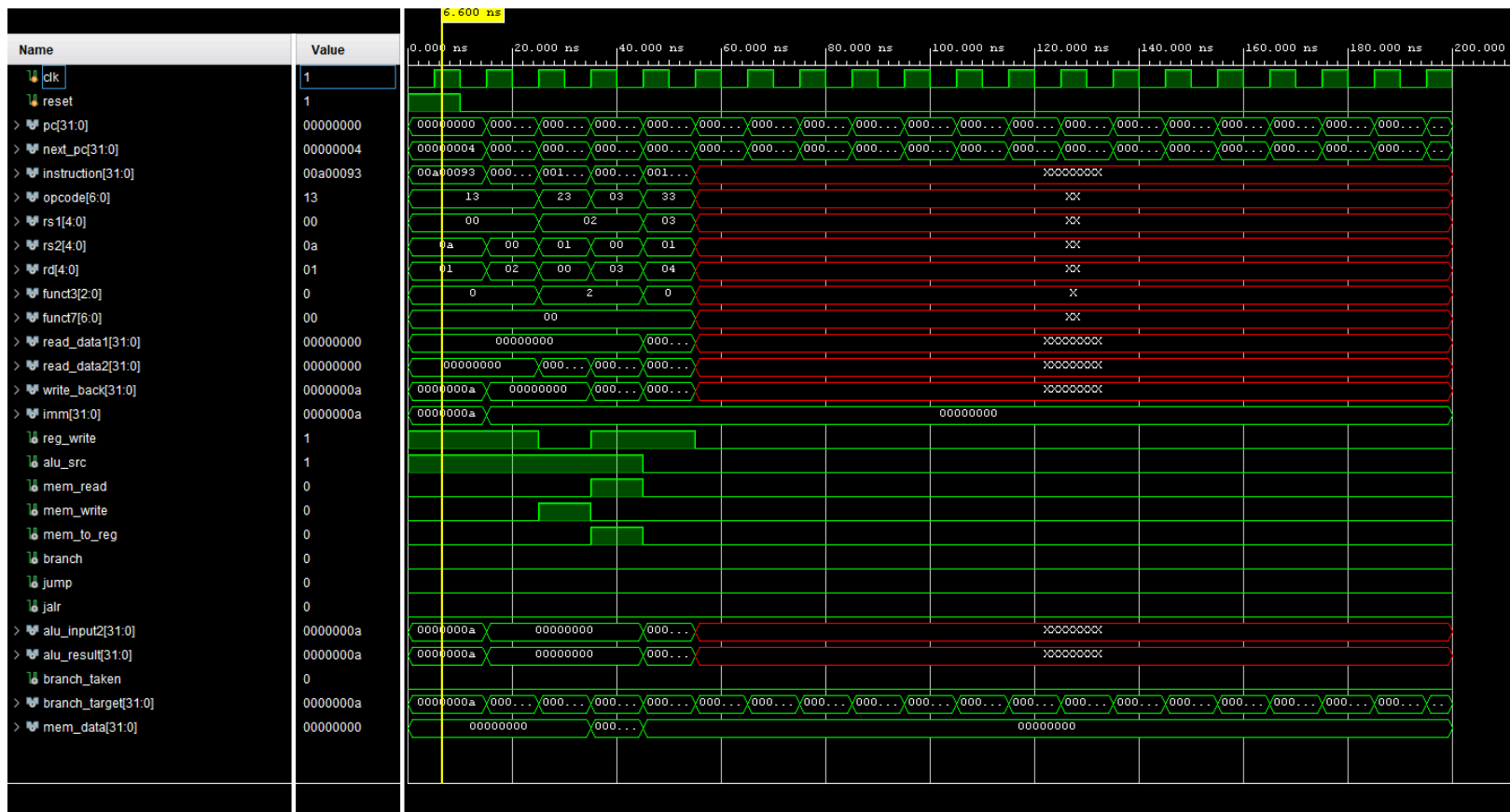
MEMORY VALUES

```
memory[0] = 10
```

```
ADDI x1,x0,10    → x1 = 10
ADDI x2,x0,0     → x2 = 0

SW x1,0(x2)      → memory[0] = 10
LW x3,0(x2)      → x3 = memory[0]

ADD x4,x3,x1     → x4 = 10 + 10
```



NOTE

Green signals (such as `clk`, `reset`, `pc`, `next_pc`, and some control signals) indicate that these parts of the processor still have **valid values and are functioning normally**, since they are generated independently by the clock and PC logic.

Red signals (like `instruction`, `opcode`, `rs1`, `rs2`, `rd`, `alu_result`) show **unknown (X) values** because the processor has **reached an address where the instruction memory has no valid instruction(all instructions executed)**, causing the decode and ALU signals that depend on the instruction to become undefined.

3.Sample HEX CODE to test **BEQ** and **BNE**.

HEX Code

00500093
00500113
00208463
00300213
00400293
00300313
FFF30313
FE031EE3
00000013

```
asm
ADDI x1,x0,5
ADDI x2,x0,5

BEQ x1,x2,label

ADDI x4,x0,3    // skipped

label:
ADDI x5,x0,4

ADDI x6,x0,3

loop:
ADDI x6,x6,-1
BNE x6,x0,loop

NOP
```

Expected Output

x1 = 5
x2 = 5
x4 = 0
x5 = 4
x6 = 0

Recieved Output :

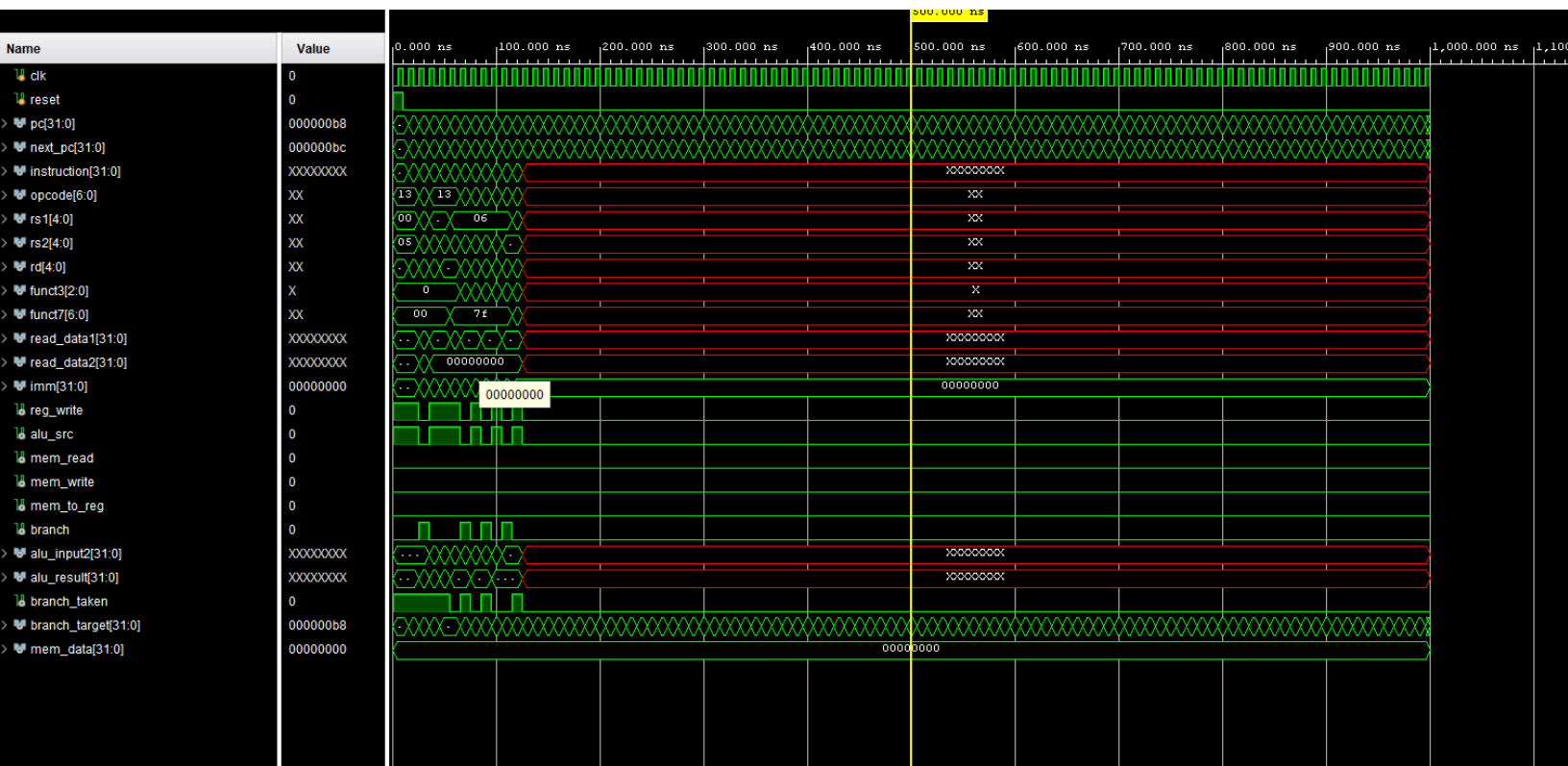
=====

FINAL REGISTER VALUES

=====

x1 = 5
x2 = 5
x3 = 0
x4 = 0
x5 = 4
x6 = 0
x7 = 0
x8 = 0

=====



4. Sample code for LOOP implementation

HEX code

```
00500093
00300113
FFF10113
FE011EE3
00000013
```

```
ADDI x1,x0,5    // x1 = 5

ADDI x2,x0,3    // x2 = 3 (loop counter)

loop:
ADDI x2,x2,-1   // decrement

BNE x2,x0,loop  // repeat until x2 = 0

NOP
```

Expected Output

x1 = 5

x2 = 0

Received Output :

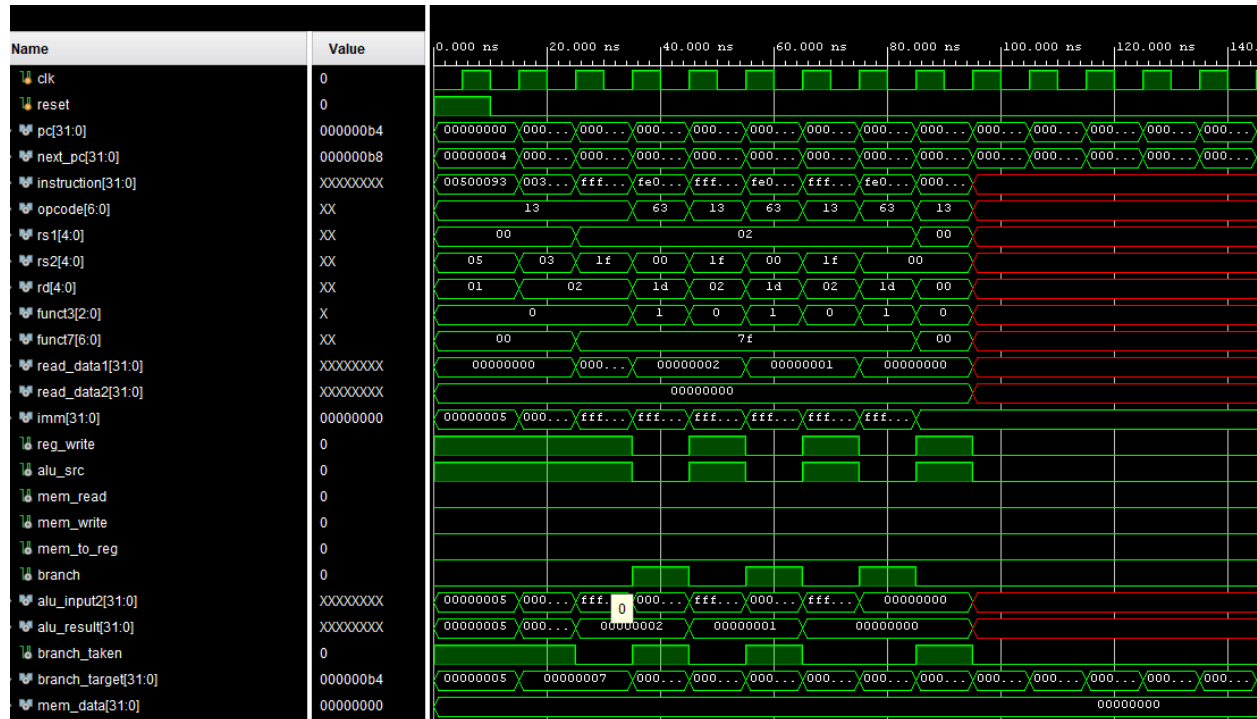
=====

FINAL REGISTER VALUES

=====

x1 = 5

x2 = 0



5. Sample code for JAL

Hex code

00500093

008000EF

00100113

00300193

00000013

```
ADDI x1,x0,5

JAL x1,label

ADDI x2,x0,1    // should be skipped

label:
ADDI x3,x0,3
```

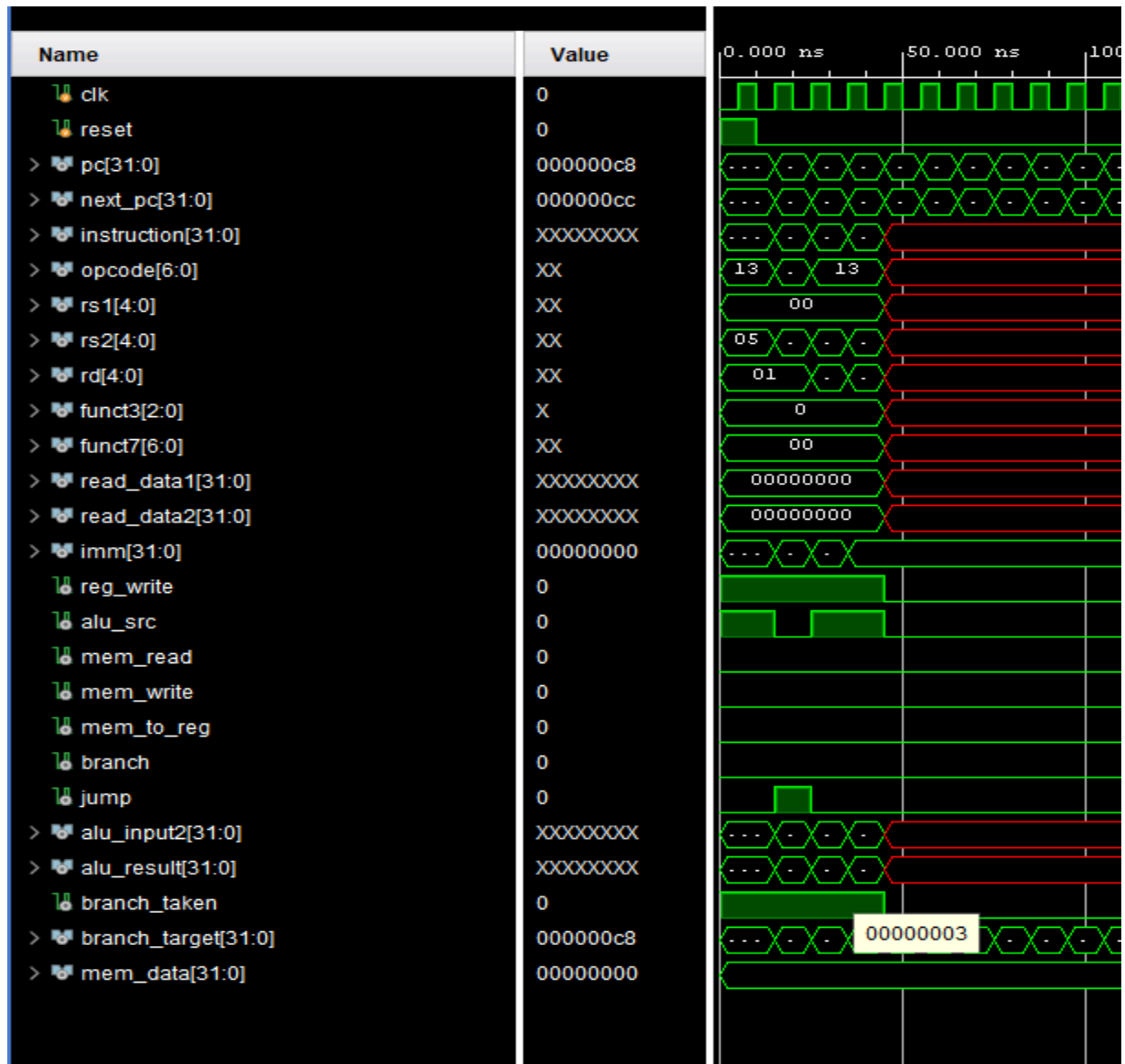
Expected Outputs

x1 = 8 ← return address
x2 = 0 ← instruction skipped
x3 = 3 ← label executed

Received Outputs

FINAL REGISTER VALUES

```
x1 =      8
x2 =      0
x3 =      3
```



6. Sample code for JALR

Hex code

00C00093

000082E7

00100113

00300193

00000013

```
ADDI x1,x0,12
JALR x5,x1,0
ADDI x2,x0,1
ADDI x3,x0,3
NOP
```

Expected Output

x1 = 12

x2 = 0

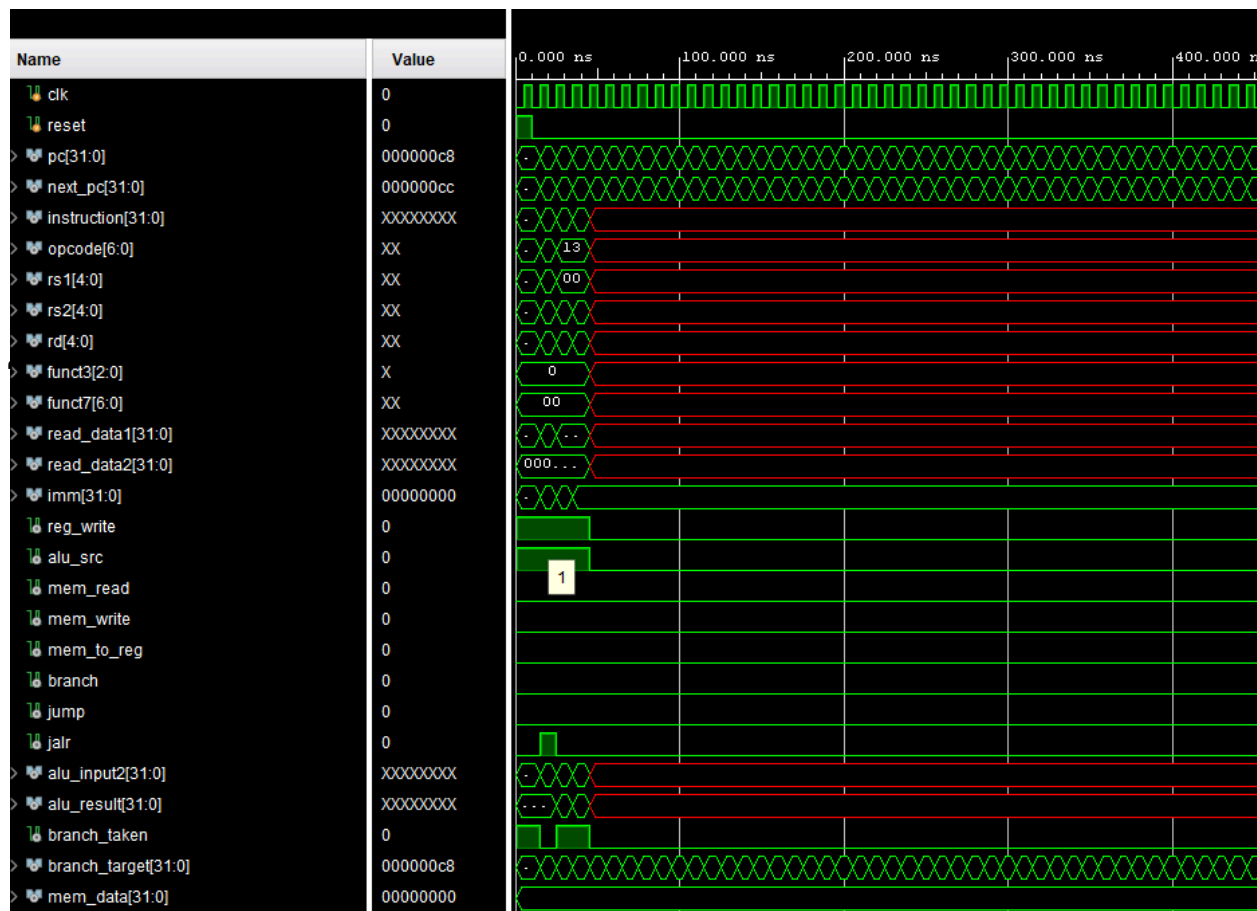
x3 = 3

x5 = 8

Received Output

FINAL REGISTER VALUES

```
x1 =      12
x2 =       0
x3 =       3
x4 =       0
x5 =       8
```



7. Sample code for Stack Usage

Hex code

```
02000113
00500093
00112E23
01C12303
00410113
00000013
```

```
ADDI x2,x0,32    # stack pointer = 32

ADDI x1,x0,5     # value to push

SW x1,28(x2)     # store at memory[60]

LW x6,28(x2)     # load from memory[60]

ADDI x2,x2,4     # restore stack pointer

NOP
```

Expected Outputs

```
x1 = 5
x2 = 36
x6 = 5
```

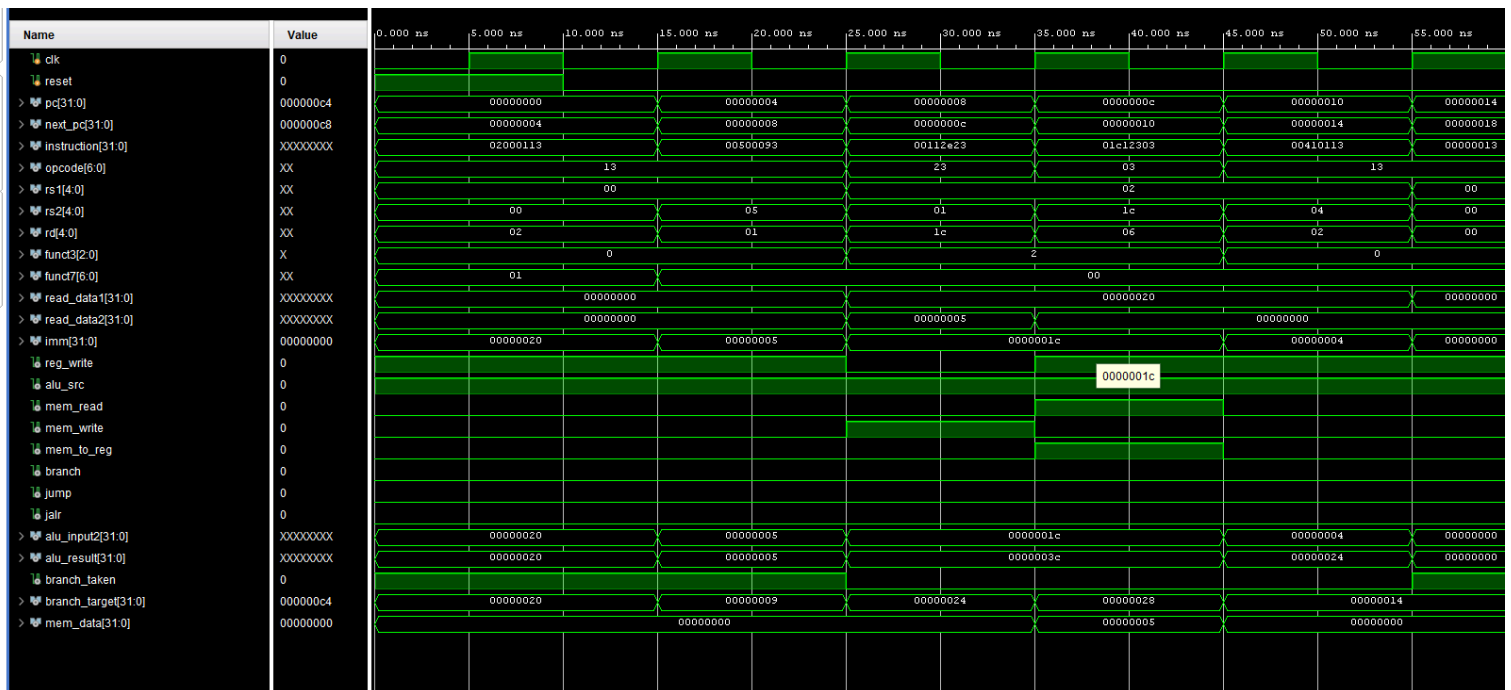
Received Outputs

=====

FINAL REGISTER VALUES

=====

```
x1 =      5
x2 =     36
x3 =      0
x4 =      0
x5 =      0
x6 =      5
```



8. Sample code for Nested Loop

Hex Code

```
00200113
00200193
00000213
FFF18193
FE019CE3
FFF10113
FE0116E3
00000013
```

```
ADDI x2,x0,2      # outer = 2
ADDI x3,x0,2      # inner = 2
ADDI x4,x0,0      # iteration counter

inner:
ADDI x3,x3,-1
BNE x3,x0,inner

ADDI x2,x2,-1
BNE x2,x0,inner

NOP
```

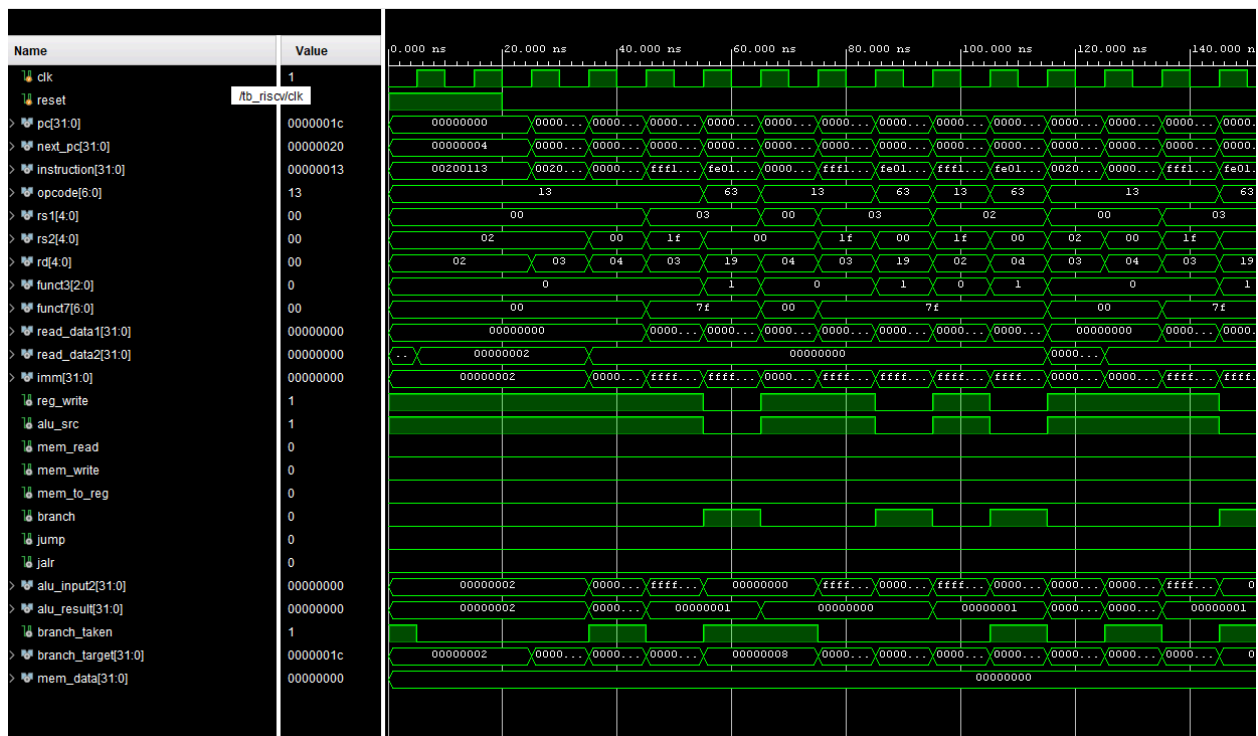
Expected Output

```
x1 = 0
x2 = 0
x3 = 0
x4 = 0
x5 = 0
x6 = 0
```

Received Output

FINAL REGISTER VALUES

```
x1 = 0
x2 = 0
x3 = 0
x4 = 0
x5 = 0
x6 = 0
x7 = 0
finish called at time : 215 ns
```



9. Sample code for Function Return

Hex Code

00500093
00C000EF
00100113
00000013
00300193
00008067

```
ADDI x1,x0,5

JAL x1,function

ADDI x2,x0,1

NOP

function:
ADDI x3,x0,3

JALR x0,x1,0
```

Expected Output

x1 = 8
x2 = 1
x3 = 3

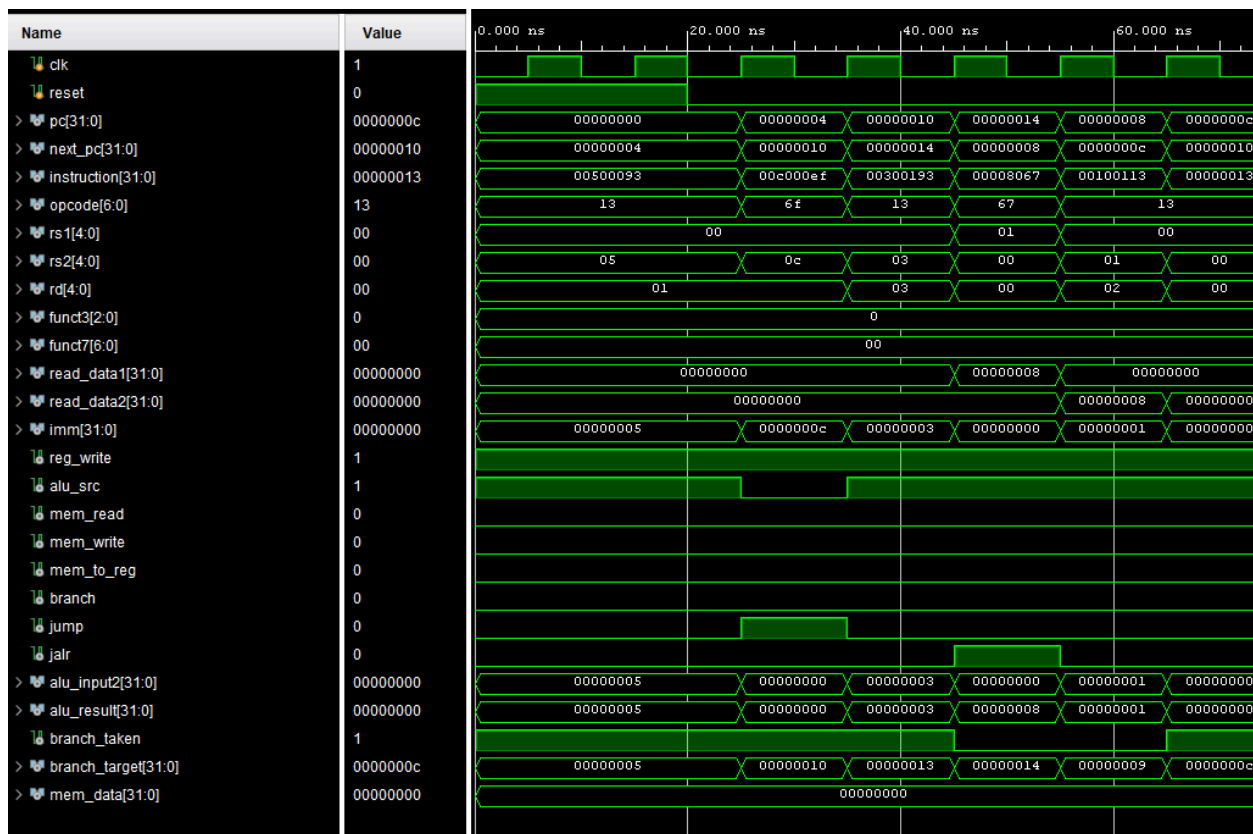
Received Output

=====

FINAL REGISTER VALUES

=====

x1 = 8
x2 = 1
x3 = 3
x4 = 0



Additional Codes

1. Sample Codes For SLTU , SRA

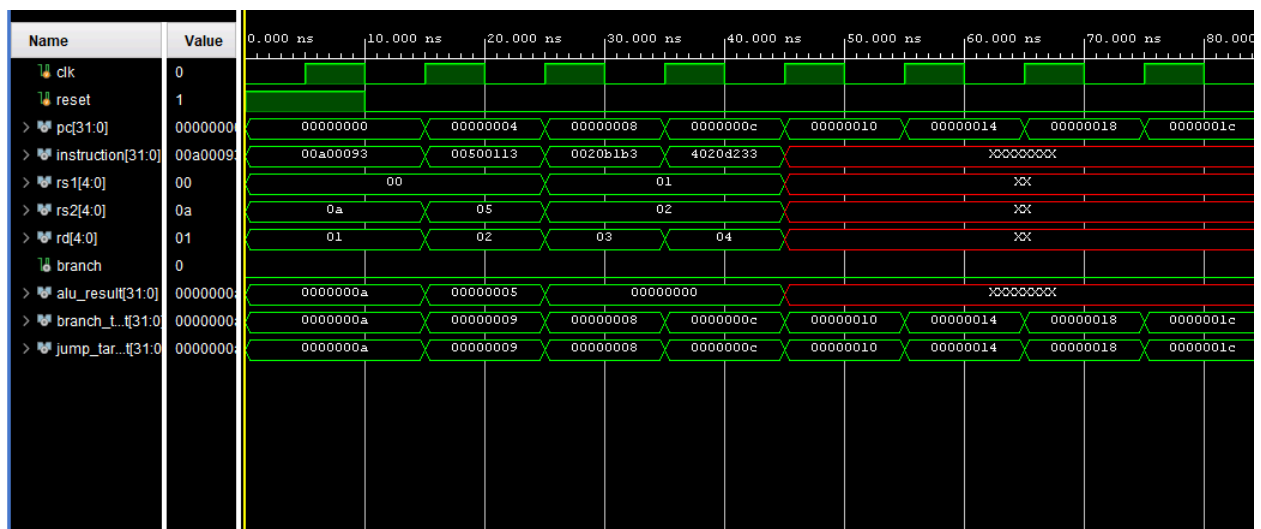
```
00A00093 // ADDI x1, x0, 10
00500113 // ADDI x2, x0, 5
0020B1B3 // SLTU x3, x1, x2
4020D233 // SRA x4, x1, x2
```

Expected:

```
x1 = 10
x2 = 5
x3 = 0 // 10 < 5 → false
x4 = 0 // 10 >> 5 = 0
```

Received :

```
=====
FINAL REGISTER VALUES
=====
x1 =      10
x2 =       5
x3 =       0
x4 =       0
x5 =       0
x6 =       0
x7 =       0
x8 =       0
=====
```



2.Sample Codes For SLTU , SRA

```
00A00093 // ADDI x1, x0, 10
```

```
0050A113 // SLTI x2, x1, 5
```

```
0050B193 // SLTIU x3, x1, 5
```

```
0050C213 // XORI x4, x1, 5
```

```
0050E293 // ORI x5, x1, 5
```

```
0050F313 // ANDI x6, x1, 5
```

Expected:

x1 = 10

x2 = 0 // 10 < 5 false

x3 = 0 // 10 < 5 unsigned false

x4 = 15 // 10 XOR 5 = 15

x5 = 15 // 10 OR 5 = 15

x6 = 0 // 10 AND 5 = 0

=====

FINAL REGISTER VALUES

=====

x1 = 10

x2 = 0

x3 = 0

x4 = 15

x5 = 15

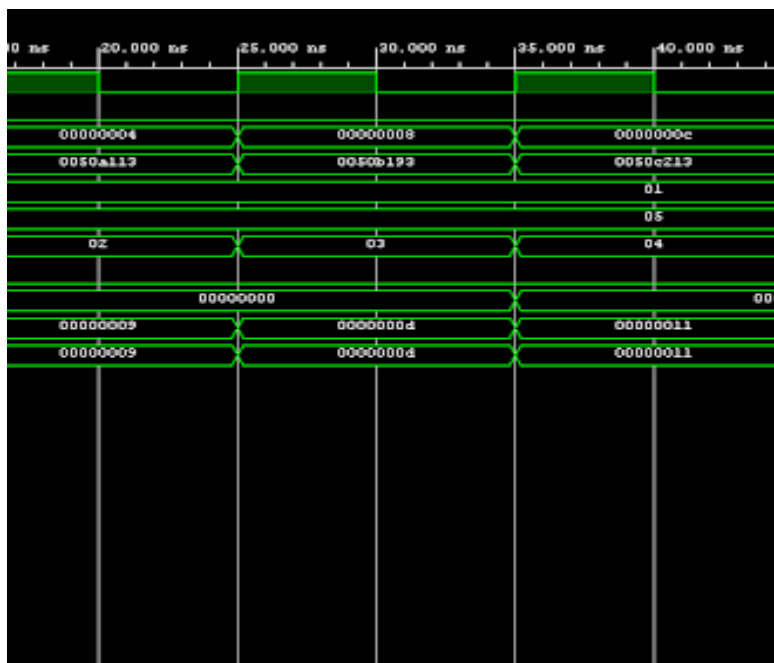
x6 = 0

x7 = 0

x8 = 0

=====

Obtained Output :



3. Sample Codes For SLLI , SLRI, SRAI

```
00A00093 // ADDI x1, x0, 10
00309113 // SLLI x2, x1, 3
0030D193 // SRLI x3, x1, 3
4030D213 // SRAI x4, x1, 3
```

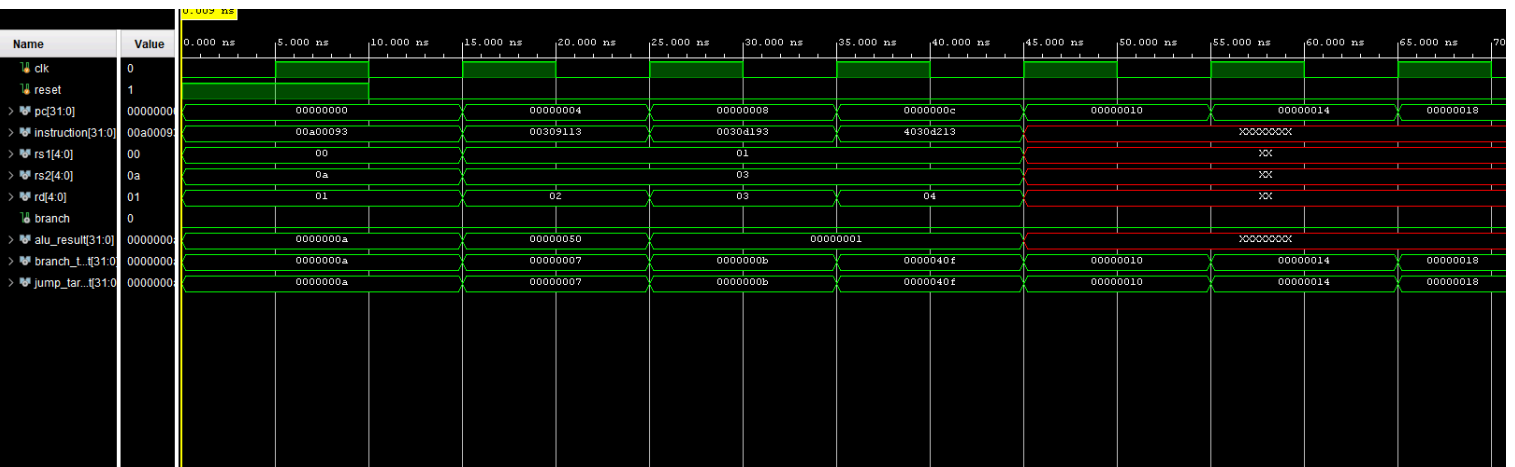
Expected:

```
x1 = 10
x2 = 80 // 10 << 3
x3 = 1 // 10 >> 3
x4 = 1 // arithmetic shift
```

Obtained Output :

FINAL REGISTER VALUES

```
x1 = 10
x2 = 80
x3 = 1
x4 = 1
x5 = 0
x6 = 0
x7 = 0
x8 = 0
```



4. Sample Codes For BLTU , BGEU

```
00A00093 // ADDI x1, x0, 10
00500113 // ADDI x2, x0, 5
0020E463 // BLTU x1, x2, +8
00100193 // ADDI x3, x0, 1
00200193 // ADDI x3, x0, 2
0020F463 // BGEU x1, x2, +8
00300213 // ADDI x4, x0, 3
```

EXPECTED:

```
x1 = 10
x2 = 5
x3 = 2 // BLTU not taken
x4 = 0 // BGEU taken so ADDI skipped
```

Obtained Output:

```
=====
M REGISTER VALUES
=====
=      10
=       5
=       2
=       0
=       0
=       0
=       0
=       0
=====
```

